

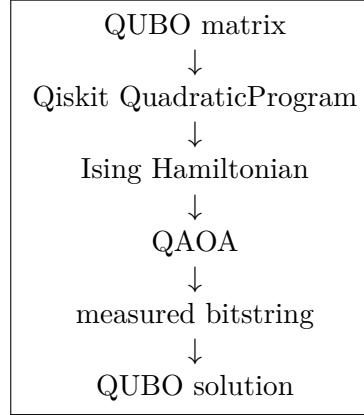
QUBO to standard QAOA using Qiskit

By: Sabah Ud Din Ahmad

Last Updated: May 30, 2026

In this document, I explain how to convert a QUBO optimization problem into a QAOA workflow using Qiskit.

The intended path is:



QUBO objective is encoded as a quantum cost Hamiltonian in QAOA.

1. Background: What problem are we solving?

A QUBO problem is a **quadratic unconstrained binary optimization** problem. It has the form

$$\min_{\mathbf{x} \in \{0,1\}^n} E_{\text{QUBO}}(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} \quad (1)$$

where

- $\mathbf{x}$: Binary decision vector.
- x_i : Binary variable, $x_i \in \{0, 1\}$.
- n : Number of binary variables.
- Q : QUBO matrix, $Q \in \mathbb{R}^{n \times n}$.
- $E_{\text{QUBO}}(\mathbf{x})$: QUBO energy/objective function.
- T : Matrix transpose.

The goal is to find the binary string

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^T Q \mathbf{x} \quad (2)$$

where

- $\mathbf{x}^*$: Best binary solution.
- $\arg \min$: Input value that gives the minimum objective.

We are searching over all possible strings of zeros and ones and choosing the one with the lowest cost.

For example, if $n = 3$, then the possible solutions are

000, 001, 010, 011, 100, 101, 110, 111.

There are 2^n possible binary strings. Therefore, brute-force search becomes expensive as n grows.

1.1 Input format

The input to the pipeline is a square numerical matrix:

$$Q = \begin{bmatrix} Q_{00} & Q_{01} & \cdots & Q_{0,n-1} \\ Q_{10} & Q_{11} & \cdots & Q_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ Q_{n-1,0} & Q_{n-1,1} & \cdots & Q_{n-1,n-1} \end{bmatrix}.$$

Required format:

```
Q = np.array([
    [q00, q01, q02],
    [q10, q11, q12],
    [q20, q21, q22],
], dtype=float)
```

Assumptions:

1. Q must be square.
2. Q must contain finite real numbers.
3. The variables are binary: $x_i \in \{0, 1\}$.
4. The optimization is a minimization problem.
5. The objective is $\mathbf{x}^T Q \mathbf{x}$.
6. If Q is not symmetric, the off-diagonal contribution is interpreted as $Q_{ij} + Q_{ji}$ for $i < j$.

Because binary variables satisfy

$$x_i^2 = x_i,$$

the diagonal entries of Q act like linear terms.

1.2 Code: Input validation

```
import numpy as np

def validate_qubo_matrix(Q: np.ndarray) -> np.ndarray:
    """
    Validate and return a QUBO matrix as a NumPy float array.

    Input:
        Q: square NumPy-compatible matrix.

    Output:
        Q: NumPy array with dtype float.
    """
    Q = np.asarray(Q, dtype=float)

    if Q.ndim != 2:
        raise ValueError("Q must be a two-dimensional matrix.")

    if Q.shape[0] != Q.shape[1]:
        raise ValueError("Q must be square, with shape (n, n).")

    if not np.all(np.isfinite(Q)):
        raise ValueError("Q must contain only finite real numbers.")
```

return Q

2. Converting QUBO to Ising Hamiltonian

QAOA does not directly optimize a classical matrix Q . QAOA optimizes a quantum cost Hamiltonian. For QUBO problems, the standard Hamiltonian is an Ising Hamiltonian:

$$\hat{H}_C = C\hat{I} + \sum_{i=0}^{n-1} h_i \hat{Z}_i + \sum_{0 \leq i < j \leq n-1} J_{ij} \hat{Z}_i \hat{Z}_j \quad (3)$$

where

- $\hat{H}_C$: QAOA cost Hamiltonian.
- C : Constant energy offset.
- $\hat{I}$: Identity operator.
- h_i : Linear Ising coefficient for qubit i .
- J_{ij} : Pairwise Ising coupling between qubits i and j .
- $\hat{Z}_i$: Pauli- Z operator acting on qubit i .
- $\hat{Z}_i \hat{Z}_j$: Two-qubit Pauli- Z interaction.

The conversion uses the binary-to-spin map

$$x_i = \frac{1 - z_i}{2} \quad (4)$$

where

- x_i : Binary QUBO variable, $x_i \in \{0, 1\}$.
- z_i : Spin variable, $z_i \in \{-1, +1\}$.

The inverse relation is

$$z_i = 1 - 2x_i \quad (5)$$

In quantum computing, the spin value z_i is represented by the eigenvalue of a Pauli- Z operator. The Pauli- Z operator satisfies

$$\begin{aligned} \hat{Z}|0\rangle &= +|0\rangle, \\ \hat{Z}|1\rangle &= -|1\rangle. \end{aligned}$$

Therefore,

- if $x_i = 0$, the quantum state is $|0\rangle$ and the Z -eigenvalue is $z_i = +1$.
- if $x_i = 1$, the quantum state is $|1\rangle$ and the Z -eigenvalue is $z_i = -1$.

So the map

$$x_i = \frac{1 - z_i}{2}$$

is consistent with computational-basis measurements.

For a general QUBO objective written as

$$E_{\text{QUBO}}(\mathbf{x}) = c + \sum_i q_i x_i + \sum_{i < j} q_{ij} x_i x_j \quad (6)$$

where

- c : Constant term in the QUBO objective.
- q_i : Linear coefficient for x_i .
- q_{ij} : Quadratic coefficient for $x_i x_j$,

substitute

$$x_i = \frac{1 - z_i}{2}.$$

The linear term becomes

$$q_i x_i = q_i \frac{1 - z_i}{2} = \frac{q_i}{2} - \frac{q_i}{2} z_i. \quad (7)$$

The quadratic term becomes

$$q_{ij} x_i x_j = q_{ij} \left(\frac{1 - z_i}{2} \right) \left(\frac{1 - z_j}{2} \right). \quad (8)$$

Expanding,

$$q_{ij} x_i x_j = \frac{q_{ij}}{4} (1 - z_i - z_j + z_i z_j). \quad (9)$$

Therefore,

$$q_{ij} x_i x_j = \frac{q_{ij}}{4} - \frac{q_{ij}}{4} z_i - \frac{q_{ij}}{4} z_j + \frac{q_{ij}}{4} z_i z_j. \quad (10)$$

Collecting terms gives

$$E_{\text{Ising}}(\mathbf{z}) = C + \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j \quad (11)$$

where

$$C = c + \frac{1}{2} \sum_i q_i + \frac{1}{4} \sum_{i < j} q_{ij} \quad (12)$$

$$h_i = -\frac{q_i}{2} - \frac{1}{4} \sum_{j \neq i} q_{ij} \quad (13)$$

$$J_{ij} = \frac{q_{ij}}{4} \quad (14)$$

where

- C : Constant offset after QUBO-to-Ising conversion.
- h_i : Linear Ising coefficient.
- J_{ij} : Quadratic Ising coupling.
- q_i : QUBO linear coefficient.
- q_{ij} : QUBO pairwise coefficient.

The constant C shifts all energies by the same amount. It does not change which bitstring is optimal.

2.1 Code: QUBO-to-Ising

```
import numpy as np
```

```
def qubo_to_ising(Q: np.ndarray, constant: float = 0.0):
    """
    Convert a QUBO matrix Q for min x^T Q x into Ising coefficients.
```

The Ising form is:

$$E(z) = C + \sum_i h[i] z_i + \sum_{\{i < j\}} J[(i, j)] z_i z_j$$

where z_i in $\{-1, +1\}$ and $x_i = (1 - z_i)/2$.

Input:

Q: square QUBO matrix.

constant: optional constant term c.

Output:

C: scalar offset.

h: NumPy array of linear Ising coefficients.

J: dictionary of pairwise Ising couplings.

"""

```
Q = validate_qubo_matrix(Q)
```

```
n = Q.shape[0]
```

```
# Diagonal QUBO entries are linear terms because  $x_i^2 = x_i$ .
```

```
q_linear = np.diag(Q).copy()
```

```
# Off-diagonal terms are combined as  $Q_{ij} + Q_{ji}$  for  $i < j$ .
```

```
q_quadratic = {}
```

```
for i in range(n):
```

```
    for j in range(i + 1, n):
```

```
        coeff = Q[i, j] + Q[j, i]
```

```
        if abs(coeff) > 1e-12:
```

```
            q_quadratic[(i, j)] = coeff
```

```
# Constant offset.
```

```
C = constant
```

```
C += 0.5 * np.sum(q_linear)
```

```
C += 0.25 * sum(q_quadratic.values())
```

```
# Linear Ising terms.
```

```
h = np.zeros(n)
```

```
for i in range(n):
```

```
    h[i] += -0.5 * q_linear[i]
```

```
for (i, j), coeff in q_quadratic.items():
```

```
    h[i] += -0.25 * coeff
```

```
    h[j] += -0.25 * coeff
```

```
# Pairwise Ising couplings.
```

```
J = {}
```

```
for (i, j), coeff in q_quadratic.items():
```

```
    J[(i, j)] = 0.25 * coeff
```

```
return C, h, J
```

3. QAOA Implementation

QAOA prepares a parameterized quantum state

$$|\psi(\gamma, \beta)\rangle = \prod_{\ell=1}^p e^{-i\beta_{\ell}\hat{H}_M} e^{-i\gamma_{\ell}\hat{H}_C} |+\rangle^{\otimes n} \quad (15)$$

where

- $|\psi(\gamma, \beta)\rangle$: QAOA trial quantum state.
- γ : Cost-Hamiltonian parameters.
- β : Mixer-Hamiltonian parameters.
- p : QAOA depth, also called the number of repetitions/layers.
- $\hat{H}_C$: Cost Hamiltonian encoding the QUBO objective.
- $\hat{H}_M$: Mixer Hamiltonian.
- $|+\rangle^{\otimes n}$: Equal superposition over all n -bit strings.

The initial state is

$$|+\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} \quad (16)$$

where

- $|+\rangle$: Single-qubit equal-superposition state.
- $\otimes$: Tensor product.

The standard mixer Hamiltonian is

$$\hat{H}_M = \sum_{i=0}^{n-1} \hat{X}_i \quad (17)$$

where

- $\hat{X}_i$: Pauli- X operator acting on qubit i .

The cost unitary is

$$U_C(\gamma_{\ell}) = e^{-i\gamma_{\ell}\hat{H}_C} \quad (18)$$

and the mixer unitary is

$$U_M(\beta_{\ell}) = e^{-i\beta_{\ell}\hat{H}_M} \quad (19)$$

A QAOA circuit alternates these two operations:

$$|+\rangle^{\otimes n} \rightarrow U_C(\gamma_1) \rightarrow U_M(\beta_1) \rightarrow \dots \rightarrow U_C(\gamma_p) \rightarrow U_M(\beta_p) \rightarrow \text{measurement}$$

QAOA starts with all possible answers in superposition. It then repeatedly applies a cost operation and a mixing operation. The classical optimizer adjusts the parameters so that low-cost bitstrings become more likely when the circuit is measured.

3.1 Code: QAOA ansatz from an Ising operator

```
from qiskit.circuit.library import QAOAAnsatz

def build_qaoa_ansatz(ising_operator, reps: int = 1):
    """
```

Build a QAOA ansatz circuit from an Ising cost operator.

Input:

ising_operator: Qiskit SparsePauliOp cost Hamiltonian.
reps: QAOA depth p.

Output:

```
ansatz: Qiskit QAOAAnsatz circuit.
"""
ansatz = QAOAAnsatz(
    cost_operator=ising_operator,
    reps=reps,
    insert_barriers=True,
)
return ansatz
```

4. Building the QUBO in Qiskit

Qiskit Optimization represents optimization problems using `QuadraticProgram`.

For a QUBO matrix Q , the objective is

$$\min_{\mathbf{x} \in \{0,1\}^n} \sum_i Q_{ii} x_i + \sum_{i < j} (Q_{ij} + Q_{ji}) x_i x_j \quad (20)$$

where

- Q_{ii} : Diagonal element of the QUBO matrix.
- $Q_{ij} + Q_{ji}$: Effective off-diagonal QUBO coefficient.
- $x_i x_j$: Product of two binary variables.

This expression is equivalent to

$$\mathbf{x}^T Q \mathbf{x}.$$

4.1 Code: Convert QUBO matrix to QuadraticProgram

```
import numpy as np
from qiskit_optimization import QuadraticProgram

def qubo_matrix_to_quadratic_program(
    Q: np.ndarray,
    name: str = "qubo_problem",
) -> QuadraticProgram:
    """
    Convert a QUBO matrix into a Qiskit QuadraticProgram.

    The objective is:

    minimize x^T Q x

    Input:
    Q: square NumPy array with shape (n, n).
    name: name of the Qiskit optimization problem.
```

```

Output:
    qp: Qiskit QuadraticProgram.
    """
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    qp = QuadraticProgram(name)

    # Create binary variables x_0, x_1, ..., x_{n-1}.
    for i in range(n):
        qp.binary_var(name=f"x_{i}")

    linear = {}
    quadratic = {}

    # Diagonal terms: Q_{ii} x_i^2 = Q_{ii} x_i.
    for i in range(n):
        if abs(Q[i, i]) > 1e-12:
            linear[f"x_{i}"] = Q[i, i]

    # Off-diagonal terms: combine Q_{ij} and Q_{ji}.
    for i in range(n):
        for j in range(i + 1, n):
            coeff = Q[i, j] + Q[j, i]
            if abs(coeff) > 1e-12:
                quadratic[(f"x_{i}", f"x_{j}")] = coeff

    qp.minimize(linear=linear, quadratic=quadratic)

    return qp

```

5. Converting the QUBO to an Ising Hamiltonian using Qiskit

Qiskit can convert a QuadraticProgram into a qubit Hamiltonian:

$$\boxed{\text{QuadraticProgram} \rightarrow \text{SparsePauliOp} + \text{offset}} \quad (21)$$

where

- `SparsePauliOp`: Qiskit sparse Pauli-operator representation of $\hat{H}_C$.
- `offset`: Constant energy shift C .
- $\hat{H}_C$: Cost Hamiltonian used by QAOA.

The relation between the QUBO energy and Ising energy is

$$\boxed{E_{\text{QUBO}}(\mathbf{x}) = E_{\text{Ising}}(\mathbf{z})} \quad (22)$$

if the offset is included. Equivalently,

$$\boxed{E_{\text{QUBO}}(\mathbf{x}) = \langle \mathbf{x} | \hat{H}_C | \mathbf{x} \rangle + C} \quad (23)$$

where

- $|\mathbf{x}\rangle$: Computational-basis quantum state corresponding to bitstring $\mathbf{x}$.
- $\langle \mathbf{x} | \hat{H}_C | \mathbf{x} \rangle$: Hamiltonian expectation value for bitstring $\mathbf{x}$.
- C : Constant offset.

5.1 Code: Qiskit QUBO-to-Ising conversion

```
def qiskit_qubo_to_ising(Q: np.ndarray):
    """
    Convert a QUBO matrix to Qiskit's Ising operator representation.

    Input:
        Q: QUBO matrix.

    Output:
        qp: QuadraticProgram.
        ising_operator: SparsePauliOp cost Hamiltonian.
        offset: constant energy offset.
    """
    qp = qubo_matrix_to_quadratic_program(Q)
    ising_operator, offset = qp.to_ising()

    return qp, ising_operator, offset
```

6. Solving QUBO with standard QAOA

QAOA is a hybrid algorithm. Quantum computer prepares and samples

$$|\psi(\gamma, \beta)\rangle.$$

The classical computer adjusts

$$\gamma, \beta$$

to reduce the expected cost:

$$F(\gamma, \beta) = \langle \psi(\gamma, \beta) | \hat{H}_C | \psi(\gamma, \beta) \rangle \quad (24)$$

where

- $F(\gamma, \beta)$: QAOA objective minimized by the classical optimizer.
- γ : Cost-layer parameters.
- β : Mixer-layer parameters.
- $\hat{H}_C$: Cost Hamiltonian.

At the end, the quantum circuit is sampled. The measured bitstrings are candidate solutions.

The final selected solution is usually the measured bitstring with the lowest original QUBO energy:

$$\mathbf{x}_{\text{best}} = \arg \min_{\mathbf{x} \in \mathcal{S}} \mathbf{x}^T Q \mathbf{x} \quad (25)$$

where

- $\mathcal{S}$: Set of bitstrings sampled from the QAOA circuit.
- $\mathbf{x}_{\text{best}}$: Best sampled QUBO solution.

6.1 Code: QAOA solver using MinimumEigenOptimizer

```
import numpy as np

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler
from qiskit_optimization.algorithms import MinimumEigenOptimizer

def solve_qubo_with_qaoa(
    Q: np.ndarray,
    reps: int = 1,
    maxiter: int = 200,
    seed: int = 123,
):
    """
    Solve a QUBO problem using Qiskit QAOA.

    Input:
    Q: square QUBO matrix.
    reps: QAOA depth p.
    maxiter: maximum number of classical optimizer iterations.
    seed: random seed for reproducibility.

    Output:
    Dictionary containing:
        - qiskit_result
        - solution
        - objective_value
        - quadratic_program
        - ising_operator
        - offset
    """
    Q = validate_qubo_matrix(Q)

    qp, ising_operator, offset = qiskit_qubo_to_ising(Q)

    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
    )

    qaoa_optimizer = MinimumEigenOptimizer(qaoa)

    result = qaoa_optimizer.solve(qp)

    solution = np.array(result.x, dtype=int)
```

```

objective_value = float(result.fval)

return {
    "qiskit_result": result,
    "solution": solution,
    "objective_value": objective_value,
    "quadratic_program": qp,
    "ising_operator": ising_operator,
    "offset": offset,
}

```

7. Example: Standard QAOA

Consider the QUBO matrix

$$Q = \begin{bmatrix} 1 & -2 & 0 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \quad (26)$$

The objective is

$$E_{\text{QUBO}}(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}. \quad (27)$$

Writing the variables explicitly:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}. \quad (28)$$

Because Q is upper triangular, the effective QUBO objective is

$$E_{\text{QUBO}}(x_0, x_1, x_2) = x_0 + x_1 + x_2 - 2x_0x_1 - 2x_1x_2 \quad (29)$$

where

- x_0, x_1, x_2 : Binary decision variables.
- E_{QUBO} : QUBO energy.

This example rewards choosing adjacent pairs (x_0, x_1) and (x_1, x_2) because the pairwise coefficients are negative.

7.1 Code

```

import numpy as np

Q = np.array([
    [1, -2, 0],
    [0, 1, -2],
    [0, 0, 1],
], dtype=float)

out = solve_qubo_with_qaoa(
    Q=Q,
    reps=1,
    maxiter=200,
    seed=123,

```

```
)

print("QAOA solution:", out["solution"])
print("QAOA objective value:", out["objective_value"])
print("Ising operator:")
print(out["ising_operator"])
print("Offset:", out["offset"])
```

Expected best solution:

$$\mathbf{x}^* = (1, 1, 1)$$

The corresponding objective value is

$$E_{\text{QUBO}}(1, 1, 1) = 1 + 1 + 1 - 2 - 2 = -1.$$

So the expected minimum is

$$E_{\min} = -1$$

8. Classical brute-force validation for standard QAOA

For small QUBO problems, the simplest validation is brute force. Brute force evaluates

$$E_{\text{QUBO}}(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$$

for every

$$\mathbf{x} \in \{0, 1\}^n.$$

The exact solution is

$$\mathbf{x}_{\text{exact}} = \arg \min_{\mathbf{x} \in \{0, 1\}^n} \mathbf{x}^T Q \mathbf{x} \quad (30)$$

where

- $\mathbf{x}_{\text{exact}}$: Exact classical minimizer.
- E_{exact} : Exact minimum QUBO energy.

This is practical only for small n , because the number of candidates is 2^n .

8.1 Code: Brute-force QUBO solver

```
import itertools
import numpy as np

def brute_force_qubo(Q: np.ndarray):
    """
    Solve a QUBO exactly by brute force.

    Input:
    Q: square QUBO matrix.

    Output:
    best_x: best binary vector.
    best_energy: minimum QUBO energy.
    all_results: list of (x, energy) pairs.
    """
```

```

Q = validate_qubo_matrix(Q)
n = Q.shape[0]

best_x = None
best_energy = np.inf
all_results = []

for bits in itertools.product([0, 1], repeat=n):
    x = np.array(bits, dtype=int)
    energy = float(x @ Q @ x)

    all_results.append((x, energy))

    if energy < best_energy:
        best_energy = energy
        best_x = x.copy()

return best_x, best_energy, all_results

```

8.2 Code: Validate standard QAOA solution against brute force

```

best_x_exact, best_energy_exact, all_results = brute_force_qubo(Q)

print("Exact solution:", best_x_exact)
print("Exact energy:", best_energy_exact)

print("QAOA solution:", out["solution"])
print("QAOA energy:", out["objective_value"])

print("Does QAOA match exact energy?")
print(np.isclose(out["objective_value"], best_energy_exact))

```

For the example above, the expected result is:

```

Exact solution: [1 1 1]
Exact energy: -1.0
QAOA solution: [1 1 1]
QAOA energy: -1.0
Does QAOA match exact energy?
True

```

Because QAOA is approximate and probabilistic, this may not always be true for larger or harder problems.

9. Validate the QUBO-to-Ising conversion

The Ising conversion is correct if every binary string gives the same energy before and after conversion.

For a bitstring $\mathbf{x}$, define

$$z_i = 1 - 2x_i \quad (31)$$

and

$$E_{\text{Ising}}(\mathbf{z}) = C + \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j \quad (32)$$

The conversion is correct if

$$E_{\text{QUBO}}(\mathbf{x}) = E_{\text{Ising}}(\mathbf{z}) \quad (33)$$

for all

$$\mathbf{x} \in \{0, 1\}^n.$$

9.1 Code: Validate Ising conversion

```
def qubo_energy(Q: np.ndarray, x: np.ndarray) -> float:
    """
    Compute QUBO energy  $x^T Q x$ .
    """
    Q = validate_qubo_matrix(Q)
    x = np.asarray(x, dtype=int)

    return float(x @ Q @ x)

def ising_energy_from_coefficients(C, h, J, x: np.ndarray) -> float:
    """
    Compute Ising energy from manual Ising coefficients.

    Input:
        C: constant offset.
        h: linear coefficients.
        J: pairwise coupling dictionary.
        x: binary vector.

    Output:
        Ising energy corresponding to x.
    """
    x = np.asarray(x, dtype=int)

    # Convert binary variables to spin variables.
    z = 1 - 2 * x

    energy = float(C)
    energy += float(np.dot(h, z))

    for (i, j), coeff in J.items():
        energy += float(coeff * z[i] * z[j])

    return energy

def validate_manual_qubo_to_ising(Q: np.ndarray, atol: float = 1e-9):
    """
    Validate that manual QUBO-to-Ising conversion preserves energy.

    Input:
        Q: QUBO matrix.
        atol: numerical tolerance.

    Output:
```

```

    True if all energies match.
    """
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    C, h, J = manual_qubo_to_ising(Q)

    for bits in itertools.product([0, 1], repeat=n):
        x = np.array(bits, dtype=int)

        e_qubo = qubo_energy(Q, x)
        e_ising = ising_energy_from_coefficients(C, h, J, x)

        if not np.isclose(e_qubo, e_ising, atol=atol):
            print("Mismatch found.")
            print("x:", x)
            print("QUBO energy:", e_qubo)
            print("Ising energy:", e_ising)
            return False

    return True

```

Usage:

```
print(validate_manual_qubo_to_ising(Q))
```

Expected output:

True

10. Validate Qiskit's Ising conversion

Qiskit returns an operator and an offset:

$$\hat{H}_C, C = \text{qp.to_ising}() \quad (34)$$

For computational-basis states,

$$E_{\text{QUBO}}(\mathbf{x}) = \langle \mathbf{x} | \hat{H}_C | \mathbf{x} \rangle + C \quad (35)$$

This gives a direct way to check whether the Qiskit conversion is consistent with the original QUBO objective.

10.1 Code: Evaluate a SparsePauliOp on a bitstring

```

def pauli_z_eigenvalue(pauli_label: str, x: np.ndarray) -> int:
    """
    Compute the eigenvalue of a Pauli string on a computational-basis bitstring.

    This assumes the Pauli string contains only I and Z characters.

    Example:
        label = "IZ"
        x = [1, 0]

```

Note:

Qiskit Pauli labels may use an ordering convention that can be confusing.

For verification, compare against Qiskit's own optimizer output and brute force.

```
"""
eigenvalue = 1

# Reverse x so the rightmost Pauli character acts on qubit 0.
x_reversed = x[::-1]

for p, bit in zip(pauli_label, x_reversed):
    if p == "Z":
        eigenvalue *= 1 if bit == 0 else -1
    elif p == "I":
        eigenvalue *= 1
    else:
        raise ValueError(f"Unexpected Pauli character: {p}")

return eigenvalue
```

```
def sparse_pauli_energy_on_bitstring(operator, x: np.ndarray, offset: float = 0.0):
    """
```

Evaluate a diagonal SparsePauliOp on a computational-basis bitstring.

Input:

operator: Qiskit SparsePauliOp containing only I and Z terms.

x: binary vector.

offset: constant offset from qp.to_ising().

Output:

Energy value.

```
"""
```

```
x = np.asarray(x, dtype=int)

energy = float(offset)

for pauli, coeff in zip(operator.paulis, operator.coeffs):
    label = pauli.to_label()
    eigenvalue = pauli_z_eigenvalue(label, x)
    energy += float(np.real(coeff)) * eigenvalue

return energy
```

10.2 Code: Validate Qiskit Ising conversion against QUBO energies

```
def validate_qiskit_ising_conversion(Q: np.ndarray, atol: float = 1e-9):
    """
```

Validate Qiskit's qp.to_ising() conversion against direct QUBO energies.

Input:

Q: QUBO matrix.
atol: numerical tolerance.

Output:

True if all bitstring energies match.

"""

Q = validate_qubo_matrix(Q)

n = Q.shape[0]

qp, ising_operator, offset = qiskit_qubo_to_ising(Q)

for bits in itertools.product([0, 1], repeat=n):

 x = np.array(bits, dtype=int)

 e_qubo = qubo_energy(Q, x)

 e_ising = sparse_pauli_energy_on_bitstring(

 operator=ising_operator,

 x=x,

 offset=offset,

)

 if not np.isclose(e_qubo, e_ising, atol=atol):

 print("Mismatch found.")

 print("x:", x)

 print("QUBO energy:", e_qubo)

 print("Qiskit Ising energy:", e_ising)

 return False

return True

Usage:

print(validate_qiskit_ising_conversion(Q))

Expected output:

True

11. Complete workflow

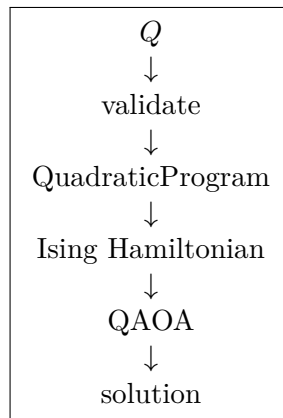
The full workflow can be expressed as the function

$$\boxed{\mathcal{A} : Q \mapsto \mathbf{x}_{\text{QAOA}}} \quad (36)$$

where

- $\mathcal{A}$: QUBO-to-QAOA algorithm.
- Q : Input QUBO matrix.
- $\mathbf{x}_{\text{QAOA}}$: Best solution returned by QAOA.

The internal steps are:



11.1 Code: Workflow class

```
import numpy as np

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler
from qiskit_optimization.algorithms import MinimumEigenOptimizer

class QUBOToQAOA:
    """
    QUBO-to-QAOA workflow.

    Input:
        Q: QUBO matrix for  $\min x^T Q x$ .

    Main methods:
        build_quadratic_program()
        convert_to_ising()
        solve()
        validate_against_brute_force()
    """

    def __init__(
        self,
        Q: np.ndarray,
        reps: int = 1,
        maxiter: int = 200,
        seed: int = 123,
    ):
        self.Q = validate_qubo_matrix(Q)
        self.reps = reps
        self.maxiter = maxiter
        self.seed = seed

        self.qp = None
        self.ising_operator = None
        self.offset = None
```

```

        self.result = None

    def build_quadratic_program(self):
        self.qp = qubo_matrix_to_quadratic_program(self.Q)
        return self.qp

    def convert_to_ising(self):
        if self.qp is None:
            self.build_quadratic_program()

        self.ising_operator, self.offset = self.qp.to_ising()
        return self.ising_operator, self.offset

    def solve(self):
        if self.qp is None:
            self.build_quadratic_program()

        if self.ising_operator is None:
            self.convert_to_ising()

        sampler = StatevectorSampler(seed=self.seed)
        optimizer = COBYLA(maxiter=self.maxiter)

        qaoa = QAOA(
            sampler=sampler,
            optimizer=optimizer,
            reps=self.reps,
        )

        qaoa_optimizer = MinimumEigenOptimizer(qaoa)
        self.result = qaoa_optimizer.solve(self.qp)

        return self.result

    def best_solution(self):
        if self.result is None:
            self.solve()

        x = np.array(self.result.x, dtype=int)
        energy = qubo_energy(self.Q, x)

        return {
            "x": x,
            "qubo_energy": energy,
            "qiskit_objective": float(self.result.fval),
        }

    def validate_against_brute_force(self):
        if self.result is None:
            self.solve()

```

```

exact_x, exact_energy, _ = brute_force_qubo(self.Q)
qaoa_solution = self.best_solution()

return {
    "exact_x": exact_x,
    "exact_energy": exact_energy,
    "qaoa_x": qaoa_solution["x"],
    "qaoa_energy": qaoa_solution["qubo_energy"],
    "energy_match": np.isclose(
        exact_energy,
        qaoa_solution["qubo_energy"],
    ),
}

```

11.2 Code: Use the workflow class

```

Q = np.array([
    [1, -2, 0],
    [0, 1, -2],
    [0, 0, 1],
], dtype=float)

workflow = QUBOToQAOA(
    Q=Q,
    reps=1,
    maxiter=200,
    seed=123,
)

workflow.solve()

print("Best QAOA solution:")
print(workflow.best_solution())

print("Validation:")
print(workflow.validate_against_brute_force())

```

12. Validation checklist

Use this summarized checklist to validate the QAOA result. This is already incorporated in the main workflow and is added here separately as a summary.

12.1 Check the QUBO matrix

Verify:

$$Q \in \mathbb{R}^{n \times n}.$$

Code:

```
Q = validate_qubo_matrix(Q)
```

12.2 Check QUBO energies directly

For a candidate solution $\mathbf{x}$, compute:

$$E_{\text{QUBO}}(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}.$$

Code:

```
x = np.array([1, 1, 1])
print(qubo_energy(Q, x))
```

12.3 Check manual Ising conversion

Verify:

$$E_{\text{QUBO}}(\mathbf{x}) = E_{\text{Ising}}(\mathbf{z})$$

for all bitstrings.

Code:

```
print(validate_manual_qubo_to_ising(Q))
```

12.4 Check Qiskit Ising conversion

Verify:

$$E_{\text{QUBO}}(\mathbf{x}) = \langle \mathbf{x} | \hat{H}_C | \mathbf{x} \rangle + C.$$

Code:

```
print(validate_qiskit_ising_conversion(Q))
```

12.5 Check QAOA against brute force for small problems

For small n , compare QAOA to exact brute force:

```
validation = workflow.validate_against_brute_force()
```

```
print(validation)
```

If

```
validation["energy_match"]
```

is `True`, then QAOA found a globally optimal bitstring for this small test.

If it is `False`, possible reasons include:

1. QAOA depth p is too small.
2. The classical optimizer got stuck.
3. The number of iterations is too low.
4. The problem is hard for the chosen ansatz.
5. The result is probabilistic and needs more repetitions or shots.
6. The QUBO encoding may have a sign or coefficient error.

13. Assumptions and limitations

13.1 QUBO assumptions

In this guide, we assume the objective is

$$\min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^\top Q \mathbf{x}.$$

If the original problem is a maximization problem,

$$\max_{\mathbf{x}} f(\mathbf{x}),$$

convert it to minimization by using

$$\min_{\mathbf{x}} -f(\mathbf{x}).$$

13.2 Constraint assumptions

In this guide, we assume the problem is unconstrained. If the original problem has constraints, such as

$$g(\mathbf{x}) = 0$$

or

$$g(\mathbf{x}) \leq 0,$$

then the constraints must be handled by one of the following methods:

1. Convert constraints into penalty terms.
2. Use Qiskit Optimization converters.
3. Use a custom mixer that preserves feasibility.
4. Solve the constrained problem classically before QAOA preprocessing.

The simplest penalty method is:

$$E_{\text{penalty}}(\mathbf{x}) = E_{\text{objective}}(\mathbf{x}) + \lambda [g(\mathbf{x})]^2 \quad (37)$$

where

- λ : Penalty weight.
- $g(\mathbf{x})$: Constraint function.

A poor penalty weight can make the QUBO invalid in practice. If λ is too small, constraints may be violated. If λ is too large, optimization can become numerically difficult.

13.3 QAOA limitations

QAOA is approximate. It does not guarantee the global optimum for arbitrary problem instances at small depth p .

Important limitations:

1. Larger p usually gives a more expressive circuit, but also increases circuit depth.
2. Deeper circuits are harder to run on noisy hardware.
3. Classical optimization can get stuck in local minima.
4. Shot noise affects measured probabilities on real backends.
5. Hardware connectivity can require extra SWAP gates.
6. The best bitstring may not be the most frequently sampled bitstring.
7. The QAOA result should be validated against the original QUBO energy, not only against the Ising energy.

13.4 Simulation versus hardware

The code above uses:

`StatevectorSampler`

This is a local exact-state simulation method.

For real IBM hardware, the sampler/backend setup must be replaced by an IBM Runtime sampler or backend workflow. Hardware execution introduces:

1. finite shots,
2. noise,
3. transpilation,
4. device connectivity constraints,
5. queue time,
6. calibration drift,
7. possible need for error mitigation.

Therefore, we should always validate the solution locally before moving to hardware.

14. Recommended project architecture

For a codebase migrating from annealing to QAOA, keep the QUBO-generation layer unchanged.

Use this architecture:

```
project/

  problem_builders/
    build_qubo.py

  solvers/
    solve_annealer.py
    solve_classical.py
    solve_qaoa_qiskit.py

  validation/
    brute_force.py
    validate_ising.py
    compare_solvers.py

  experiments/
    run_example_qubo.py
```

The key abstraction should be:

```
def solve_qubo(Q, method="qaoa"):
    if method == "qaoa":
        return solve_qubo_with_qaoa(Q)

    elif method == "classical":
        return brute_force_qubo(Q)

    elif method == "annealer":
```

```

        return solve_with_quantum_annealer(Q)

    else:
        raise ValueError(f"Unknown method: {method}")

```

This lets the same QUBO matrix be sent to:

1. a classical solver,
2. a quantum annealer,
3. Qiskit QAOA,
4. future IBM Runtime workflows.

15. Complete script

This is the copy-paste version.

```

import itertools
import numpy as np

from qiskit_optimization import QuadraticProgram
from qiskit_optimization.algorithms import MinimumEigenOptimizer

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA

from qiskit.primitives import StatevectorSampler

def validate_qubo_matrix(Q):
    Q = np.asarray(Q, dtype=float)

    if Q.ndim != 2:
        raise ValueError("Q must be a two-dimensional matrix.")

    if Q.shape[0] != Q.shape[1]:
        raise ValueError("Q must be square.")

    if not np.all(np.isfinite(Q)):
        raise ValueError("Q must contain only finite real numbers.")

    return Q

def qubo_energy(Q, x):
    Q = validate_qubo_matrix(Q)
    x = np.asarray(x, dtype=int)
    return float(x @ Q @ x)

def qubo_matrix_to_quadratic_program(Q, name="qubo_problem"):
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    qp = QuadraticProgram(name)

```



```

for i in range(n):
    qp.binary_var(name=f"x_{i}")

linear = {}
quadratic = {}

for i in range(n):
    if abs(Q[i, i]) > 1e-12:
        linear[f"x_{i}"] = Q[i, i]

for i in range(n):
    for j in range(i + 1, n):
        coeff = Q[i, j] + Q[j, i]
        if abs(coeff) > 1e-12:
            quadratic[(f"x_{i}", f"x_{j}")] = coeff

qp.minimize(linear=linear, quadratic=quadratic)

return qp

def brute_force_qubo(Q):
    Q = validate_qubo_matrix(Q)
    n = Q.shape[0]

    best_x = None
    best_energy = np.inf

    for bits in itertools.product([0, 1], repeat=n):
        x = np.array(bits, dtype=int)
        energy = qubo_energy(Q, x)

        if energy < best_energy:
            best_energy = energy
            best_x = x.copy()

    return best_x, best_energy

def solve_qubo_with_qaoa(Q, reps=1, maxiter=200, seed=123):
    Q = validate_qubo_matrix(Q)
    qp = qubo_matrix_to_quadratic_program(Q)
    ising_operator, offset = qp.to_ising()
    sampler = StatevectorSampler(seed=seed)
    optimizer = COBYLA(maxiter=maxiter)

    qaoa = QAOA(
        sampler=sampler,
        optimizer=optimizer,
        reps=reps,
    )

    qaoa_optimizer = MinimumEigenOptimizer(qaoa)

```

```

result = qaoa_optimizer.solve(qp)

x_qaoa = np.array(result.x, dtype=int)
e_qaoa = qubo_energy(Q, x_qaoa)

return {
    "solution": x_qaoa,
    "qubo_energy": e_qaoa,
    "qiskit_objective": float(result.fval),
    "ising_operator": ising_operator,
    "offset": offset,
    "raw_result": result,
}

if __name__ == "__main__":
    Q = np.array([
        [1, -2, 0],
        [0, 1, -2],
        [0, 0, 1],
    ], dtype=float)
    exact_x, exact_energy = brute_force_qubo(Q)
    qaoa_out = solve_qubo_with_qaoa(
        Q=Q,
        reps=1,
        maxiter=200,
        seed=123,
    )

    print("Exact solution:", exact_x)
    print("Exact energy:", exact_energy)

    print("QAOA solution:", qaoa_out["solution"])
    print("QAOA QUBO energy:", qaoa_out["qubo_energy"])

    print("Energy match:")
    print(np.isclose(exact_energy, qaoa_out["qubo_energy"]))

```

16. References

1. Qiskit Optimization documentation, QuadraticProgram
2. Qiskit Optimization documentation, MinimumEigenOptimizer
3. Qiskit Optimization tutorial, minimum eigen optimizer and QUBO-to-Ising conversion
4. IBM Quantum documentation, QAOAAnsatz
5. IBM Quantum documentation, StatevectorSampler
6. IBM Quantum QAOA tutorial